

Detecting Illicit Transactions in Cryptocurrency Networks Using Graph Neural Networks

Emma Houston
Georgia Institute of Technology
eroerty3@gatech.edu

Saidat Ibiribigbe
Georgia Institute of Technology
sibiribigbe@gatech.edu

Ethan Vaz Falcao
Georgia Institute of Technology
efalcao3@gatech.edu

Yatri Patel
Georgia Institute of Technology
ypatel1306@gatech.edu

Abstract

The rise of cryptocurrency has introduced significant challenges in detecting illicit financial activities such as money laundering and fraud. Traditional machine learning approaches treat transactions as independent entities, failing to capture the relational structure inherent in blockchain data. In this work, we apply Graph Neural Networks (GNNs) — specifically GraphSAGE, Graph Convolutional Networks and Graph Attention Networks, to the Elliptic Bitcoin dataset to exploit both transaction-level features and network topology for fraud detection. We evaluate models on node classification and link prediction tasks, using F1-score and ROC-AUC to account for class imbalance, and compare against XGBoost baselines. We further investigate cross-blockchain transfer learning by evaluating models trained on Bitcoin data against Ethereum transaction data, assessing the generalizability of learned patterns across decentralized financial systems.

1. Introduction

The rapid growth of cryptocurrency adoption has led to a rapid increase in number of illicit financial activities, including money laundering, fraud, and the financing of criminal enterprises. Blockchain networks such as Bitcoin and Ethereum process millions of transactions daily, which offers a great opportunity for automated fraud detection, but the sheer scale and complexity of these networks make identifying suspicious activity a challenge.

To address this, we use Graph Neural Networks (GNNs) which operate on graph-structured data, allowing models to incorporate both the features of individual transactions and the topology of the network. GNNs can capture multi-hop behavioral patterns that traditional models miss. In

this work, we use the Elliptic Bitcoin dataset [4], a real-world labeled transaction graph, to train and evaluate both baseline and graph-based models for fraud detection. We implement Random Forest and XGBoost as a feature-only baseline and compare them against three GNN architectures: Graph Convolutional Network (GCN), GraphSAGE, and Graph Attention Networks (GAT) for node classification to identify illicit transactions. Additionally, we extend GraphSAGE to a link prediction, where the goal shifts from labeling individual transactions to inferring suspicious relationships between them, surfacing hidden connections that node-level classification may miss. This work demonstrates that graph-based approach improve fraud detection in decentralized financial systems, with direct implications for regulators, compliance teams, and the broader effort to secure decentralized finance.

2. Related Works

Historically, fraud detection in financial systems relies on traditional machine learning methods such as logistic regression, random forests, and gradient boost approaches like XGBoost. These models are trained on transaction-level features and treat each transaction independently. While they are effective in conventional cases, they ignore the relational structure of the network, which is the heart of how cryptocurrency data flows.

Weber et al. [17] were among the first to apply a Graphical structure to blockchain data, on Elliptic dataset, and achieved a F1 score of 0.628 for illicit transactions using GCNs against logistic regression and random forest baselines, establishing a reference point for subsequent work. More recently, Asiri and Somasundaram [1] compared GCN against several traditional and deep learning models, reinforcing what [17] proposed. Beyond fraud detection, Kasula et al. [11] demonstrated that GCNs can be

applied more broadly to classify blockchain addresses into different categories, highlighting versatility of graph-based methods. These along with related explorations in GNN-based blockchain fraud detection [10], collectively demonstrate GNNs as state of the art for relational financial data.

A common limitation in previous work is the focus on GCN as the primary graph architecture with node classification as the sole task. Methodologically, GraphSAGE [8] introduced inductive neighborhood sampling, while GAT [16] added attention-based neighbor weighting that can be advantageous when neighbors contribute unequally. Severe class imbalance is typically handled with simple reweighting. In our work, we benchmark GraphSAGE and GAT alongside GCN on Elliptic and additionally extend GraphSAGE to link prediction, which is largely unexplored in prior Elliptic-based studies.

3. Data

For this paper, we used the Elliptic Bitcoin dataset [4, 17], which is a publicly available transaction graph derived from the Bitcoin blockchain and labeled by Elliptic. The dataset is distributed through Kaggle [5] and consists of three files: a node feature matrix, an edge list, and a class label file, that were downloaded via Kaggle CLI. Pre-processing was applied to produce the artifacts that models used to maintain consistency.

This dataset contains 203,769 nodes (transactions) and 234,355 directed edges (flow). Of these, 4,545 nodes (2.2%) are illicit (scams, ransomware, etc.), 42,019 (20.6%) licit (exchanges, wallet providers, miners, etc.), and 157,205 (77.1%) unknown, reflecting a realistic scenario. We treat edges as undirected since GNN neighborhood aggregation is direction-agnostic. To address the class imbalance (roughly 1:9 illicit-to-licit ratio among labeled nodes), some models apply graph-aware SMOTE augmentation [3] to the training partition only, leaving validation and test splits untouched.

Table 1: Class distribution (train, validation, and test splits)

Split	Licit	Illicit	Total
Train	29,413	3,181	32,594
Validation	6,303	682	6,985
Test	6,303	682	6,985
Total	42,019	4,545	46,564

Each node has 165 anonymized, standardized numerical features: 94 local (inputs, outputs, fees, output volume, aggregated BTC statistics) and 71 one-hop neighborhood features. We apply a `StandardScaler` and impute infinities or missing values with the per-column median. The data spans 49 time steps, but we treat it as non-temporal.

We construct train/validation/test splits across the full time range over labeled nodes using a 70/15/15 ratio, stratified on the binary label so the illicit class is proportionally represented in all three splits. This yields 32,594 training, 6,985 validation, and 6,985 test nodes, with class counts in Table 1. A temporal split would more closely model deployment conditions, but a random split lets us compare architectures on a fixed benchmark without confounding results with concept drift.

4. Approach

4.1. Transaction Classification

4.1.1 Traditional Machine Learning

To establish a good baseline for fraud detection, we apply traditional supervised machine learning to transaction-level tabular data. The idea is to treat each transaction as independent and predict whether it is fraudulent or not. We compare two baseline models: Random Forest and XGBoost.

Random Forest is an ensemble method that builds multiple decision trees using bootstrapped samples of training data, then aggregates their predictions by majority voting. This reduces variance and makes the model more stable, especially on noisy tabular data. We use `class_weight = "balanced"` to handle class imbalance by weighting the minority class (fraud) more heavily during training. We also train an **XGBoost** (Extreme Gradient Boosting) model that builds trees sequentially, with each new tree trying to improve on the errors from previous ones. This technique usually performs well on complex and imbalanced tabular datasets. We use `scale_pos_weight` to address class imbalance by increasing the importance of the minority class during training.

Both models are trained on the same transaction-level features and evaluated on the test set. Since the data is heavily imbalanced, accuracy is not a reliable metric, so we use ROC-AUC and Average Precision as primary evaluation metrics, as they better capture performance across decision thresholds. We also compute precision, recall, and F1-score to better understand performance on illicit transactions.

Class imbalance is the main concern since fraudulent transactions are far less common than non-fraudulent ones. The models trained without major issues, but accuracy is misleading, hence we focused on ROC-AUC and Average Precision. Handling imbalance via weighting was necessary to make the evaluation results meaningful.

These models serve as non-graph baselines that rely only on engineered transaction features, providing a reference point for evaluating whether incorporating graph structure in later models improves fraud detection performance.

4.1.2 Graph Convolutional Networks

Graph Convolutional Networks, introduced by Kipf and Welling [13] perform node classification by aggregating features from a node’s local neighborhood using a fixed symmetric Laplacian normalization ($D^{-1/2}AD^{-1/2}$). Unlike GAT, which learns attention weights per neighbor, GCN treats all neighbors equally during aggregation. We include GCN as a middle ground between the feature-only baselines and the more expressive GNN architectures it incorporates graph structure but without the added complexity of attention or inductive sampling.

We implement a three-layer GCN using GCNConv from PyTorch Geometric [6], with ELU activations and dropout between layers. The first two layers map input features to a hidden representation, and the third layer produces two output logits for the licit and illicit classes. Raw logits are passed directly to the loss function, which is important, as applying any activation like LogSigmoid before CrossEntropyLoss causes the loss to receive doubly log-scaled values and the model fails to learn.

To handle class imbalance, we use focal loss with effective number reweighting, passing the actual training class counts (29,413 licit, 3,181 illicit) to compute per-class weights via the reweight function. We train with Adam using a ReduceLROnPlateau scheduler over 150 epochs, with gradient clipping to stabilize training. Hyperparameters were tuned via grid search over hidden channels, dropout, learning rate, weight decay, and focal loss γ and β .

The best configuration used hidden channels of 128, dropout of 0.6, learning rate of 0.001, weight decay of 0.001, $\gamma = 2$, and $\beta = 0.9999$. On the test set, the model achieved an illicit F1 of 0.57, precision of 0.45, recall of 0.77, and ROC-AUC of 0.92. The relatively high recall (0.77) shows the model is catching most illicit transactions, but the lower precision (0.45) means it also flags a fair number of licit ones as suspicious. This precision-recall tradeoff is consistent with the other GNN models prioritizing recall is the right choice in fraud detection since missing a real illicit transaction is more costly than a false alarm.

4.1.3 Graph Attention Networks (GATs)

Graph Attention Networks (GATs) introduced in 2018, were designed to leverage self-attention layers to address the shortcomings of existing graph networks. GATs stack multiple graph attention layers, each of which computes an attention coefficient matrix for each node and its neighbors [16]. Similar to 2D convolutional layers, as graph attention layers stack, the network’s receptive field grows so that weights influence more of the graph at deeper layers. As layers stack, nodes can “see” neighbors that are further away in terms of hops, allowing them to capture deeper structural relationships within the graph [16]. GATs

are worth exploring for this dataset because they can learn richer relationships between nodes to better fit the data. For each node, GAT assigns its neighbors an importance weight, which is useful for analyzing how transactions interact on the blockchain [16].

The main problem that we anticipated was the class imbalance. To handle this, we used transductive learning which creates the graph from the entire dataset, not just the training set. These models can only make predictions on data points that are already present in the graph at training time [18]. We implemented this by masking out the validation and test sets during training, then predicting on them at inference time. Transductive learning allows those nodes to contribute their relationships and connections while holding out the labels during training. Since we cannot afford to remove nodes from graph, this is especially beneficial for small or highly imbalanced datasets, like the Elliptic dataset. It allows us to get the full picture of the graph without fitting to the validation and test sets.

For this we mirrored the architecture from [16]’s inductive learning experiments, which was a three-layer model with ELU activation layers between each. We also tested the transductive architecture outlined in the paper, but the three layer version was a better fit, as it resulted in the better F1 score [16]. Starting with similar parameters, we conducted a grid search over various optimization, loss, and regularization hyper-parameters to maximize the illicit F1 score of the validation set resulting in a final model.

A problem that we had to address was the memory issues resulting from loading the entire graph into memory at once. This is a symptom of the transductive approach, and it made finetuning time consuming and limited, in that we had to limit the combination of number of attention heads and hidden channels to prevent memory explosion. With more memory, we believe that we could have created a better fit model, however, we anticipate only marginal improvements for the cost.

4.1.4 GraphSAGE

The key idea behind GraphSAGE is that we learn how to aggregate feature information from a node’s local neighborhood [8], making it well-suited for transaction graphs where new nodes appear over time. This differs from GCN, which weights neighbors using a fixed symmetric Laplacian normalization, and GAT, which uses learned attention weights. GraphSAGE instead applies a learnable aggregator (mean, in this scenario) and keeps the node’s representation separate from the aggregated neighbor representation. Together, these design choices give GraphSAGE a different inductive bias than GCN and GAT, motivating its inclusion as one of the three GNN architectures we evaluate on the Elliptic transaction graph.

We implement a two-layer GraphSAGE model [8] using the SAGEConv operator from PyTorch Geometric [6]. The first convolutional layer maps the 165 input node features to a hidden dimension, followed by ReLU activation and dropout [15] for regularization, and the second layer projects the hidden representation to two output logits for the licit and illicit classes. The hidden dimension and dropout rate are tuned as hyperparameters. We train the Elliptic dataset using the splits described in Section 3, retaining unknown-labeled nodes (encoded as -1) so that their features contribute to their labeled neighbors, while excluding them from loss and evaluation by filtering on labels ≥ 0 .

To address class imbalance, we apply two mechanisms. First, we compute inverse-frequency class weights [12], $w_c = N/(C \cdot n_c)$, where N is the total number of labeled training samples, $C = 2$, and n_c is the count of class c . Second, we use focal loss [14] instead of cross-entropy, which down-weights well-classified examples and concentrates learning on hard, often misclassified, illicit cases. We also experimented with graph-aware SMOTE augmentation [3], applied only to the training partition. Models are trained with Adam, producing logits for all nodes via two rounds of neighborhood aggregation at each epoch. We monitor validation F1 on the illicit class and retain the weights from the best-performing epoch, since F1 is more informative than loss or accuracy for imbalanced binary classification [9].

To identify a strong configuration, we run a YAML-driven hyperparameter search: the full Cartesian product of the search space is enumerated, shuffled with a fixed seed, and truncated to a trial budget, yielding a randomized search over the grid rather than exhaustive evaluation [2]. The search space spans hidden channel size, dropout, learning rate, weight decay, training epochs, focal loss γ and β , and the SMOTE flag. The trial with the highest validation F1 is selected as the final model.

4.2. Link Prediction

For link prediction, we construct a graph from the transaction data and split it into training, validation and test sets using *RandomLinkSplit*. Positive and negative edges are generated during this split using a fixed negative sampling ratio of 1 (1:1 positive-to-negative edges). Using this technique avoids extreme class imbalance during training since real graph data is usually sparse.

We use a GraphSAGE model with two convolutional layers, ReLU activation and dropout to learn the node embeddings. GraphSAGE was used because it learns node representations by aggregating information from a node’s local neighbourhood. We also use a dot-product decoder to compute scores between the node pairs for edge prediction. The model is trained using binary cross-entropy (BCE) loss with logits. During training, we used both positive and negative edges from the training split directly.

The performance was evaluated using ROC-AUC and Average Precision on the validation and test sets, since these metrics work better for imbalanced ranking problems. We also compute F1-score by searching over classification thresholds, but the best model selection is based on Average Precision. The best performing model on the validation is stored and then evaluated on the test set.

A limitation of this setup is that the negative edges are sampled randomly, which may not fully reflect the difficulty of separating realistic fraudulent relationships in the graph. In addition, the model operates on a static graph and does not capture temporal ordering of transactions, which is often important in financial fraud settings. However, this approach provides a stable baseline for evaluating link prediction performance.

5. Experiments and Results

For the **baseline models**, we train **Random Forest** and **XGBoost** on the same feature set and evaluate them on a held-out test split. Both models are designed to handle class imbalance: Random Forest uses `class_weight="balanced"`, while XGBoost uses `scale_pos_weight` to increase the contribution of the minority (illicit) class during training. Since the dataset is highly imbalanced, we rely on ROC-AUC and Average Precision as the main evaluation metrics, with additional reporting of F1-score for the illicit class.

XGBoost achieves the strongest performance among the tabular models, with a ROC-AUC of 0.997 and Average Precision of 0.986, slightly outperforming Random Forest. This suggests that boosting-based sequential correction of errors is better suited to capturing complex decision boundaries in the feature space compared to bagging.

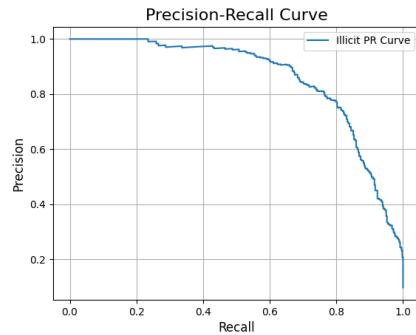


Figure 1: GAT Test Precision-Recall Curve

To analyze the performance of the **GAT** model, we used both the precision-recall curve (1), and the F1 score (2). As mentioned above, the model is tuned on the illicit F1 metric. Since our goal is to identify illicit transactions to prevent criminal behavior, a higher true positive rate is beneficial

even at the cost of more false positives. However, there is a tradeoff with precision: maximizing recall risks classifying everything as illicit, which is costly. Therefore we use the precision-recall curve and F1 to find the best balance. As shown in Table 2, we were able to achieve a 0.77 F1-score for the illicit class over 100 epochs of training. This is due to the use of focal loss and the specification of raw class size to the loss function. Focal loss allows us to re-weight the classes based on their effective numbers, placing more importance on harder to classify data points. Adding this helps us overcome the 1:9 illicit-to-licit ratio in the training data. A recall of 0.83 means we are correctly labeling 83% of all the illicit transaction. Though this comes at the cost of a slightly lower precision, it would be safer to have more false positives if it means finding more true positives.

For **GraphSAGE**, as mentioned in Section 4.1.4, we trained a two-layer model using focal loss with inverse frequency class weights to handle the imbalance. We ran a randomized search of 36 trials over hidden channels $\in \{64, 128, 256\}$, dropout $\in \{0.3, 0.5\}$, learning rate $\in \{5e-4, 1e-3, 1e-2\}$, weight decay $\in \{1e-4, 5e-4\}$, and focal loss $\gamma \in \{1, 2, 3\}$ (with $\beta = 0.9999$ fixed). Each trial trained for 150 epochs, with the best configuration selected by validation F1 on the illicit class.

The sweep showed strong sensitivity to learning rate. The seven trials at $lr = 0.01$ produced a mean validation F1 of 0.735, compared to 0.420 at 10^{-3} and 0.277 at $5 \cdot 10^{-4}$. Focal loss γ also clearly separated: $\gamma = 2$ achieved 0.527, versus 0.458 ($\gamma = 3$) and 0.297 ($\gamma = 1$). Hidden channel size had a smaller effect with mean validation F1 of 0.368, 0.447, and 0.466 for 64, 128, and 256 channels. Lower dropout (0.3) modestly outperformed 0.5 with a mean of 0.458 vs 0.407.

The best configuration (hidden size 128, dropout 0.3, learning rate 0.01, weight decay $5 \cdot 10^{-4}$, $\gamma = 2$), achieved validation F1 of 0.788 and test F1 of 0.793 on the illicit class, with a test ROC-AUC of 0.975. Across the full search, the mean validation-to-test F1 gap was +0.011 with a range of $[-0.026, +0.050]$.

We initially ran the search with 100 epochs but observed validation F1 still rising at epoch 97 (best 0.750), so we extended to 150 epochs under identical hyperparameters and seeds, the best configuration improved from validation/test F1 of 0.750/0.754 to 0.788/0.793. The training curves (Figure 2) show train and validation loss track closely with no signs of overfitting. Validation F1 was zero for the first ten epochs as the focal-loss-trained model initially predicted licit for all nodes, then climbed steadily before flattening, gaining roughly 0.01 over the final 30 epochs.

The precision-recall curve for the best model (Figure 3) achieves an average precision of 0.911 on the test set, versus a chance-level baseline of 0.098. Precision remains essentially perfect out to recall 0.66, then degrades gracefully,

staying above 0.85 at recall 0.8. Validation and test curves overlap closely (AP 0.904 vs 0.911). In the per-class breakdown (Table ??), GraphSAGE achieved precision 0.99 and recall 0.66 on the illicit class at the default threshold, with an F1 of 0.79.

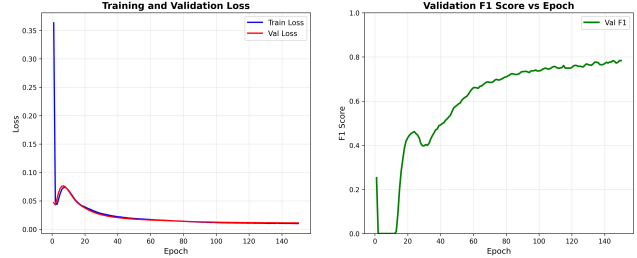


Figure 2: GraphSAGE training and validation loss (left) and validation F1 score on the illicit class (right)

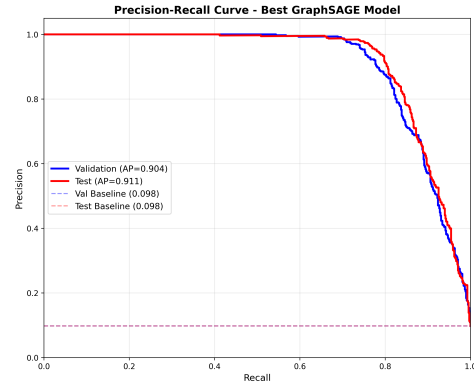


Figure 3: GraphSAGE precision-recall curve

For **link prediction**, we construct a transaction graph and train a GraphSAGE model using a 1:1 negative sampling ratio via RandomLinkSplit. The model is trained to distinguish between observed (positive) and sampled (negative) edges using a dot-product decoder and binary cross-entropy loss. We tune several hyperparameters, including hidden dimension, learning rate, and dropout via grid search and select the best one to run on our test set based on its validation Average Precision.

The model achieves a ROC-AUC of 0.873 and Average Precision of 0.858 and an F1-Score of 0.81 on the test set, which is lower than the tabular baselines. Despite this, when we looked at the edge score distribution (Figure 4), we saw a clear distinction between positive (real) and negative (fake) edges. The positive edges had higher predicted probabilities, which is a good sign, indicating that the model was able to differentiate between observed and sampled edges. However, GraphSAGE still performed worse than the tabular models like XGBoost.

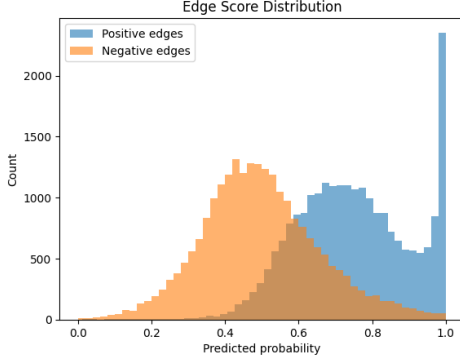


Figure 4: Link Prediction - Edge Score Distribution

These results show that while the graph structure provides useful information for link prediction, the transaction-level features from the tabular models are still more predictive for this dataset. GraphSAGE was able to capture relationships between transactions, but XGBoost and Random Forest, which rely on structured feature data, still outperformed it. This shows that both graph structure and feature-based signals in fraud detection are complementary and should be used together.

Table 2: Classification Results

Classifier	Class	Precision	Recall	F1-Score	Support
XGBoost	Licit	0.99	1.00	1.00	6303
	Illicit	0.98	0.94	0.96	682
Random Forest	Licit	0.99	0.99	0.99	6303
	Illicit	0.96	0.91	0.93	682
GCN	Licit	0.97	0.90	0.93	6303
	Illicit	0.45	0.77	0.57	682
GAT	Licit	0.98	0.97	0.97	4205
	Illicit	0.72	0.83	0.77	455
GraphSAGE	Licit	0.96	1.00	0.98	6303
	Illicit	0.99	0.66	0.79	682

6. Discussion

Overall, our results show that both graph-based and feature-based approaches have clear strengths and weaknesses for detecting illicit cryptocurrency transactions, and that they work best when thought of as complementary rather than competing.

The tabular baselines, XGBoost and Random Forest, performed surprisingly well, achieving F1 scores of 0.96 and 0.93 on the illicit class respectively. This makes sense given that the Elliptic dataset was specifically engineered with rich transaction-level features. However, these models

treat every transaction as independent, which means they completely miss the relational patterns in the graph.

Among the GNN models, GAT and GraphSAGE both showed meaningful results. GAT achieved an illicit F1 of 0.77 with a recall of 0.83, meaning it correctly identified 83% of all illicit transactions. GraphSAGE achieved an illicit F1 of 0.79 with very high precision (0.99) but lower recall (0.66). The tradeoff between these two is worth noting: in a real fraud detection setting, recall matters more than precision since missing an illicit transaction is worse than flagging a legitimate one. GCN achieved an illicit F1 of 0.57 with a recall of 0.77, which is a reasonable result given its simpler aggregation mechanism compared to GAT and GraphSAGE.

The link prediction results show that GraphSAGE can also detect suspicious relationships between transactions, achieving a ROC-AUC of 0.873. While this is lower than the node classification results, it captures a different and complementary signal, suspicious connections rather than suspicious individual transactions.

The generalization test showed that none of our models transfer well from Bitcoin to Ethereum [7]. The GraphSAGE and GCN achieved an F1 of 0.0 on Ethereum, which is not surprising given that Ethereum only had 7 native features compared to the 165 in the Elliptic dataset, and had to be padded with 0s, resulting in no meaningful data on the Ethereum side. This result highlights that the learned representations are tightly tied to Bitcoin-specific feature distributions and graph topology, and cross-blockchain generalization remains an open and difficult problem.

7. Conclusion

In this project, we explored using Graph Neural Networks for detecting illicit transactions in cryptocurrency networks. We implemented and evaluated three GNN architectures: GCN, GAT, and GraphSAGE, on the Elliptic Bitcoin dataset, alongside XGBoost and Random Forest baselines, and extended GraphSAGE to a link prediction setting.

Our main takeaways were that tabular baselines are hard to beat when domain-specific features are available, but they miss relational patterns that GNNs can capture. Also, among GNN models, GraphSAGE had the best overall F1 on the illicit class, while GAT had the best recall. Finally that, focal loss with effective number reweighting was essential across all GNN models to avoid the model just predicting everything as licit. Fourth, cross-blockchain generalization is still a hard problem, models trained on Bitcoin did not transfer to Ethereum in any meaningful way.

Future work could explore temporal graph models that respect the ordering of transactions, domain adaptation techniques for cross-blockchain transfer, and combining node-level and link-level fraud signals into a single unified model.

References

- [1] Abdullah Asiri and K. Somasundaram. Graph convolution network for fraud detection in bitcoin transactions. *Scientific Reports*, 15(11076), 2025. 1
- [2] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012. 4
- [3] Rok Blagus and Lara Lusa. Smote for high-dimensional class-imbalanced data. *BMC Bioinformatics*, 14(106), 2013. 2, 4
- [4] Elliptic. Elliptic — crypto asset risk management. <https://www.elliptic.co>. 1, 2
- [5] Elliptic. Elliptic data set. <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>, 2019. 2
- [6] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. 3, 4
- [7] Google Cloud. Ethereum Blockchain Public Dataset on BigQuery. <https://console.cloud.google.com/marketplace/product/ethereum/crypto-ethereum-blockchain>, 2018. bigquery-public-data.crypto_ethereum, accessed 2026-05-03. 6
- [8] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.02216. 2, 3, 4
- [9] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. 4
- [10] Wisdom K. Hotor. Blockchain-based fraud detection using graph neural networks (gnn’s). Master’s thesis, Berlin School of Business and Innovation, 2025. 2
- [11] Vamshi Krishna Kasula, Akhila Reddy Yadulla, Mounika Yenugula, Bhavani Konda, and Sandeep Ayyamgari. Blockchain address classification using graph neural networks for enhanced security and analytics. *Discover Artificial Intelligence*, 5:285, 2025. 1
- [12] Gary King and Langche Zeng. Logistic regression in rare events data. *Political Analysis*, 9(2):137–163, 2001. 4
- [13] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907. 3
- [14] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. 4
- [15] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014. 4
- [16] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. *International Conference on Learning Representations*, 2018. 2, 3
- [17] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019. 1, 2
- [18] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. Revisiting semi-supervised learning with graph embeddings. *Proceedings of the 33rd International Conference on Machine Learning*, 48, 2016. 3

A. Work Division

Student Name	Contributed Aspects	Details
Emma Houston	GAT Creation, Experimentation, Analysis	Implemented a GAT and trained from scratch on the El-liptic dataset. Tuned the hyperparameters to maximize performance.
Ethan Vaz Falcao	GCN, Experimentation, Analysis	Implemented a GCN and trained from scratch on the El-liptic dataset. Tuned the hyperparameters to maximize performance
Saidat Ibiribigbe	Baseline Model and Link Prediction	Implemented Random Forest, Extreme Gradient Boosting (XGBoost), and GraphSAGE for Link Prediction. Tuned model hyperparameters to optimize performance and evaluate results.
Yatri Patel	Data Preprocessing, GraphSAGE Experimentation & Analysis, Ethereum Generalization Test	Implemented data ingestion, preprocessing, and cleaning. Built and fine-tuned GraphSAGE for node classification. Developed the Ethereum cross-blockchain generalization test script and analyzed transfer performance.

Table 3: Contributions of team members.